

Enterprise Document Archiving System

Technology Stack

Backend

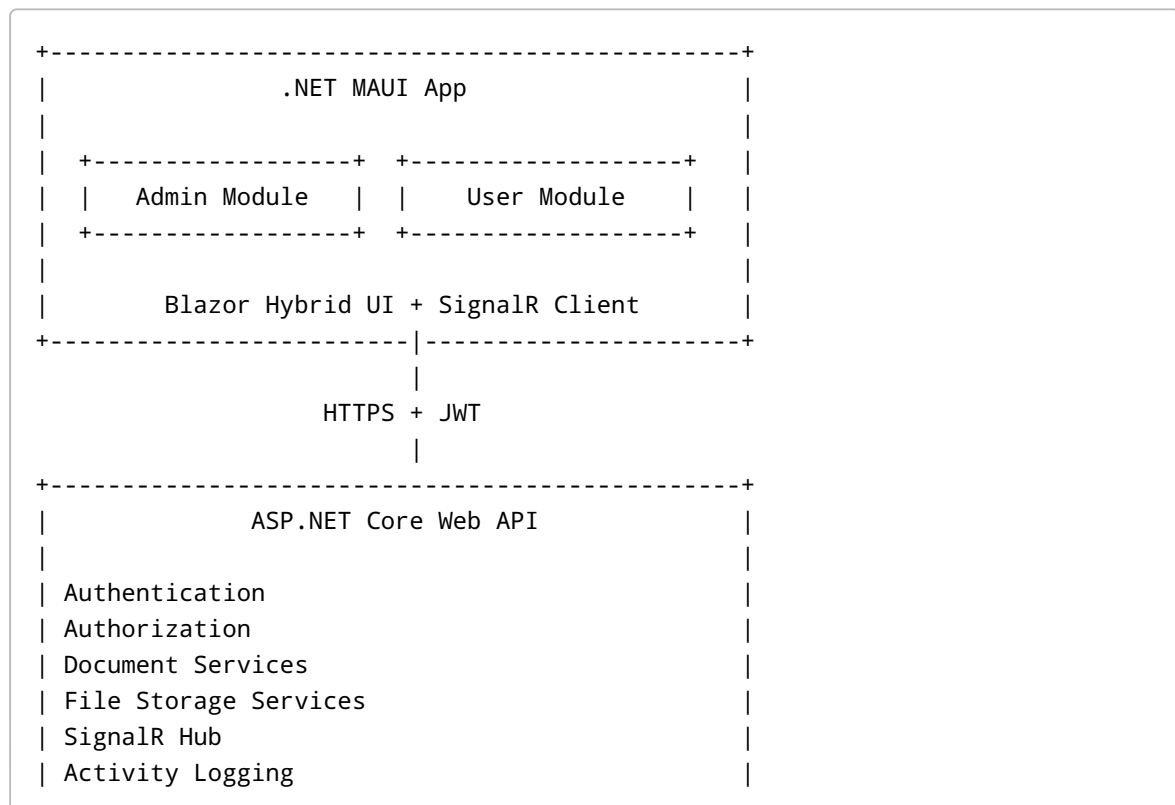
- ASP.NET Core Web API (.NET 9 recommended)
- Entity Framework Core
- SQL Server
- SignalR
- JWT Authentication
- Clean Architecture
- Repository + Unit of Work Pattern

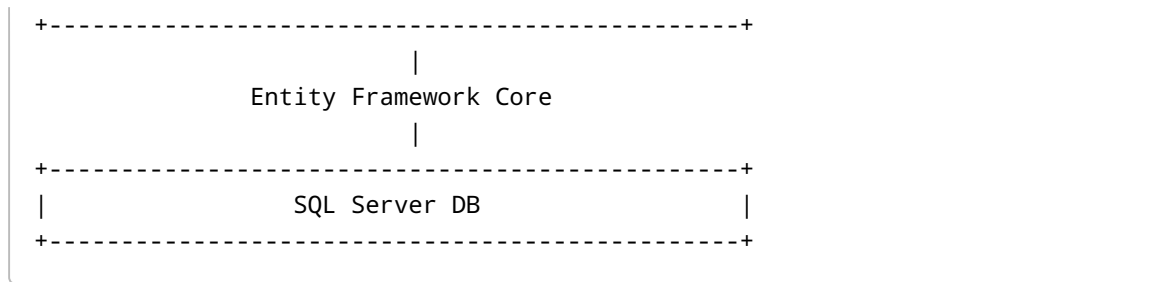
Frontend

- .NET MAUI Blazor Hybrid
- MudBlazor UI Components
- SignalR Client
- Secure Local Storage

1. Solution Architecture

High-Level Architecture





2. Clean Architecture Structure

Solution Structure

DocumentArchivingSystem.sln

```
src/
├── BuildingBlocks/
│   ├── Shared/
│   ├── Contracts/
│   └── Common/
├── Core/
│   ├── Domain/
│   └── Application/
├── Infrastructure/
│   ├── Persistence/
│   ├── Identity/
│   ├── FileStorage/
│   ├── SignalR/
│   └── Logging/
├── Presentation/
│   ├── Api/
│   └── MauiBlazor/
└── Tests/
    ├── UnitTests/
    └── IntegrationTests/
```

3. Domain Layer

Entities

User.cs

Location:

Core/Domain/Entities/User.cs

```
public class User
{
    public Guid Id { get; set; }
    public string FullName { get; set; } = default!;
    public string Email { get; set; } = default!;
    public string PasswordHash { get; set; } = default!;
    public bool IsActive { get; set; }

    public Guid RoleId { get; set; }
    public Role Role { get; set; } = default!;

    public ICollection<Document> Documents { get; set; }
        = new List<Document>();
}
```

Role.cs

```
public class Role
{
    public Guid Id { get; set; }
    public string Name { get; set; } = default!;

    public ICollection<Permission> Permissions { get; set; }
        = new List<Permission>();
}
```

Permission.cs

```
public class Permission
{
    public Guid Id { get; set; }
```

```
        public string Name { get; set; } = default!;  
    }  
}
```

Folder.cs

```
public class Folder  
{  
    public Guid Id { get; set; }  
    public string Name { get; set; } = default!;  
  
    public Guid? ParentFolderId { get; set; }  
    public Folder? ParentFolder { get; set; }  
  
    public ICollection<Folder> Children { get; set; }  
        = new List<Folder>();  
  
    public ICollection<Document> Documents { get; set; }  
        = new List<Document>();  
}
```

Document.cs

```
public class Document  
{  
    public Guid Id { get; set; }  
  
    public string FileName { get; set; } = default!;  
    public string StoredFileName { get; set; } = default!;  
    public string ContentType { get; set; } = default!;  
    public long FileSize { get; set; }  
  
    public Guid UploadedById { get; set; }  
    public User UploadedBy { get; set; } = default!;  
  
    public Guid FolderId { get; set; }  
    public Folder Folder { get; set; } = default!;  
  
    public DateTime UploadedAt { get; set; }  
}
```

ActivityLog.cs

```
public class ActivityLog
{
    public Guid Id { get; set; }

    public Guid UserId { get; set; }
    public string Action { get; set; } = default!;
    public string Description { get; set; } = default!;

    public DateTime Timestamp { get; set; }
}
```

4. Database Design

SQL Server Database Script

```
CREATE DATABASE DocumentArchivingDb;
GO

USE DocumentArchivingDb;
GO

CREATE TABLE Roles
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL
);

CREATE TABLE Permissions
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    Name NVARCHAR(100) NOT NULL
);

CREATE TABLE RolePermissions
(
    RoleId UNIQUEIDENTIFIER NOT NULL,
    PermissionId UNIQUEIDENTIFIER NOT NULL,

    PRIMARY KEY(RoleId, PermissionId),

    FOREIGN KEY(RoleId) REFERENCES Roles(Id),
    FOREIGN KEY(PermissionId) REFERENCES Permissions(Id)
);

CREATE TABLE Users
```

```

(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    FullName NVARCHAR(200) NOT NULL,
    Email NVARCHAR(200) UNIQUE NOT NULL,
    PasswordHash NVARCHAR(MAX) NOT NULL,
    IsActive BIT NOT NULL,

    RoleId UNIQUEIDENTIFIER NOT NULL,

    FOREIGN KEY(RoleId) REFERENCES Roles(Id)
);

CREATE TABLE Folders
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,
    Name NVARCHAR(200) NOT NULL,

    ParentFolderId UNIQUEIDENTIFIER NULL,

    FOREIGN KEY(ParentFolderId) REFERENCES Folders(Id)
);

CREATE TABLE Documents
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,

    FileName NVARCHAR(255) NOT NULL,
    StoredFileName NVARCHAR(255) NOT NULL,
    ContentType NVARCHAR(255) NOT NULL,
    FileSize BIGINT NOT NULL,

    UploadedById UNIQUEIDENTIFIER NOT NULL,
    FolderId UNIQUEIDENTIFIER NOT NULL,

    UploadedAt DATETIME2 NOT NULL,

    FOREIGN KEY(UploadedById) REFERENCES Users(Id),
    FOREIGN KEY(FolderId) REFERENCES Folders(Id)
);

CREATE TABLE ActivityLogs
(
    Id UNIQUEIDENTIFIER PRIMARY KEY,

    UserId UNIQUEIDENTIFIER NOT NULL,
    Action NVARCHAR(200) NOT NULL,
    Description NVARCHAR(MAX) NOT NULL,

    Timestamp DATETIME2 NOT NULL,

```

```
        FOREIGN KEY(UserId) REFERENCES Users(Id)
    );
```

5. Entity Framework Core

AppDbContext

Location:

```
Infrastructure/Persistence/AppDbContext.cs
```

```
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options)
    {
    }

    public DbSet<User> Users => Set<User>();
    public DbSet<Role> Roles => Set<Role>();
    public DbSet<Document> Documents => Set<Document>();
    public DbSet<Folder> Folders => Set<Folder>();
    public DbSet<ActivityLog> ActivityLogs => Set<ActivityLog>();

    protected override void OnModelCreating(ModelBuilder builder)
    {
        base.OnModelCreating(builder);

        builder.Entity<Folder>()
            .HasOne(x => x.ParentFolder)
            .WithMany(x => x.Children)
            .HasForeignKey(x => x.ParentFolderId)
            .OnDelete(DeleteBehavior.Restrict);
    }
}
```

6. Repository Pattern

Generic Repository Interface

```
public interface IRepository<T>
    where T : class
```

```
{
    Task<T?> GetByIdAsync(Guid id);
    Task<IEnumerable<T>> GetAllAsync();
    Task AddAsync(T entity);
    void Update(T entity);
    void Delete(T entity);
}
```

Generic Repository Implementation

```
public class Repository<T> : IRepository<T>
    where T : class
{
    protected readonly AppDbContext _context;

    public Repository(AppDbContext context)
    {
        _context = context;
    }

    public async Task AddAsync(T entity)
    {
        await _context.Set<T>().AddAsync(entity);
    }

    public void Delete(T entity)
    {
        _context.Set<T>().Remove(entity);
    }

    public async Task<IEnumerable<T>> GetAllAsync()
    {
        return await _context.Set<T>().ToListAsync();
    }

    public async Task<T?> GetByIdAsync(Guid id)
    {
        return await _context.Set<T>().FindAsync(id);
    }

    public void Update(T entity)
    {
        _context.Set<T>().Update(entity);
    }
}
```

7. JWT Authentication

JWT Configuration

appsettings.json

```
"Jwt": {  
  "Key": "SUPER_SECRET_ENTERPRISE_KEY",  
  "Issuer": "DocumentArchiveApi",  
  "Audience": "DocumentArchiveClients",  
  "DurationInMinutes": 120  
}
```

JWT Service

```
public interface IJwtService  
{  
    string GenerateToken(User user);  
}
```

```
public class JwtService : IJwtService  
{  
    private readonly IConfiguration _configuration;  
  
    public JwtService(IConfiguration configuration)  
    {  
        _configuration = configuration;  
    }  
  
    public string GenerateToken(User user)  
    {  
        var claims = new List<Claim>  
        {  
            new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),  
            new Claim(ClaimTypes.Email, user.Email),  
            new Claim(ClaimTypes.Role, user.Role.Name)  
        };  
  
        var key = new SymmetricSecurityKey(  
            Encoding.UTF8.GetBytes(_configuration["Jwt:Key"]!));  
  
        var credentials = new SigningCredentials(  
            key,  
            SecurityAlgorithms.HmacSha256);
```

```

        var token = new JwtSecurityToken(
            issuer: _configuration["Jwt:Issuer"],
            audience: _configuration["Jwt:Audience"],
            claims: claims,
            expires: DateTime.UtcNow.AddHours(2),
            signingCredentials: credentials);

        return new JwtSecurityTokenHandler().WriteToken(token);
    }
}

```

8. Authentication API

Login DTO

```

public class LoginRequestDto
{
    public string Email { get; set; } = default!;
    public string Password { get; set; } = default!;
}

```

Auth Controller

```

[ApiController]
[Route("api/auth")]
public class AuthController : ControllerBase
{
    private readonly IUserService _userService;
    private readonly IJwtService _jwtService;

    public AuthController(
        IUserService userService,
        IJwtService jwtService)
    {
        _userService = userService;
        _jwtService = jwtService;
    }

    [HttpPost("login")]
    public async Task<ActionResult> Login(LoginRequestDto request)
    {
        var user = await _userService.ValidateUserAsync(
            request.Email,
            request.Password);
    }
}

```

```

        if (user == null)
            return Unauthorized();

        var token = _jwtService.GenerateToken(user);

        return Ok(new
        {
            token
        });
    }
}

```

9. Role-Based Authorization

Policy Configuration

Program.cs

```

builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly",
        policy => policy.RequireRole("Admin"));

    options.AddPolicy("UserOnly",
        policy => policy.RequireRole("User"));
});

```

Protected Controller

```

[Authorize(Roles = "Admin")]
[ApiController]
[Route("api/users")]
public class UsersController : ControllerBase
{
}

```

10. Document Upload Service

File Storage Structure

```
Storage/  
├── Documents/  
│   ├── 2026/  
│   └── 2027/
```

Secure File Upload

```
public async Task<string> SaveFileAsync(IFormFile file)  
{  
    var extension = Path.GetExtension(file.FileName);  
  
    var allowedExtensions = new[]  
    {  
        ".pdf",  
        ".docx",  
        ".xlsx",  
        ".png",  
        ".jpg"  
    };  
  
    if (!allowedExtensions.Contains(extension.ToLower()))  
        throw new Exception("Invalid file type.");  
  
    var storedFileName =  
        $"{Guid.NewGuid()}{extension}";  
  
    var path = Path.Combine(  
        "Storage",  
        "Documents",  
        DateTime.UtcNow.Year.ToString(),  
        storedFileName);  
  
    Directory.CreateDirectory(Path.GetDirectoryName(path)!);  
  
    using var stream = new FileStream(path, FileMode.Create);  
    await file.CopyToAsync(stream);  
  
    return storedFileName;  
}
```

11. SignalR Real-Time Updates

SignalR Hub

```
public class NotificationHub : Hub
{
    public async Task SendDocumentUploaded(string message)
    {
        await Clients.All.SendAsync(
            "DocumentUploaded",
            message);
    }
}
```

Register SignalR

```
builder.Services.AddSignalR();
```

```
app.MapHub<NotificationHub>("/notificationHub");
```

MAUI SignalR Client

```
private HubConnection? _hubConnection;

public async Task InitializeAsync()
{
    _hubConnection = new HubConnectionBuilder()
        .WithUrl("https://localhost:5001/notificationHub")
        .WithAutomaticReconnect()
        .Build();

    _hubConnection.On<string>(
        "DocumentUploaded",
        (message) =>
        {
            Console.WriteLine(message);
        });

    await _hubConnection.StartAsync();
}
```

12. API Endpoints

Authentication

```
POST    /api/auth/login
POST    /api/auth/register
POST    /api/auth/refresh
```

Users

```
GET      /api/users
GET      /api/users/{id}
POST     /api/users
PUT      /api/users/{id}
DELETE   /api/users/{id}
```

Documents

```
GET      /api/documents
GET      /api/documents/{id}
POST     /api/documents/upload
DELETE   /api/documents/{id}
```

Folders

```
GET      /api/folders
POST     /api/folders
PUT      /api/folders/{id}
DELETE   /api/folders/{id}
```

Activity Logs

```
GET      /api/activitylogs
```

13. Activity Logging

Logging Service

```
public interface IActivityLogger
{
    Task LogAsync(Guid userId,
                  string action,
                  string description);
}
```

```
public class ActivityLogger : IActivityLogger
{
    private readonly AppDbContext _context;

    public ActivityLogger(AppDbContext context)
    {
        _context = context;
    }

    public async Task LogAsync(
        Guid userId,
        string action,
        string description)
    {
        var log = new ActivityLog
        {
            Id = Guid.NewGuid(),
            UserId = userId,
            Action = action,
            Description = description,
            Timestamp = DateTime.UtcNow
        };

        await _context.ActivityLogs.AddAsync(log);
        await _context.SaveChangesAsync();
    }
}
```

14. .NET MAUI Blazor Hybrid

MAUI Structure

```
Presentation/MauiBlazor/  
|  
├─ Components/  
├─ Pages/  
├─ Services/  
├─ Shared/  
├─ Layout/  
├─ Authentication/  
└─ wwwroot/
```

15. Dashboard UI

Dashboard.razor

```
<MudGrid>  
  <MudItem xs="12" md="4">  
    <MudCard>  
      <MudCardContent>  
        <MudText>Total Documents</MudText>  
        <MudText Typo="Typo.h4">1500</MudText>  
      </MudCardContent>  
    </MudCard>  
  </MudItem>  
</MudGrid>
```

16. File Explorer UI

Features

- Expandable folder tree
 - Drag-and-drop upload
 - Search
 - Filter by category
 - Breadcrumb navigation
 - Real-time updates
-

Suggested Components

MudTreeView
MudTable
MudBreadcrumbs
MudDialog
MudFileUpload

17. Access Level Permissions

Permission Matrix

Feature	Admin	User
Create User	Yes	No
Delete Document	Yes	Limited
Upload Document	Yes	Yes
View Logs	Yes	No
Manage Settings	Yes	Limited

18. Enterprise Security Best Practices

Backend Security

- Use HTTPS only
- Validate MIME types
- Scan uploaded files
- Restrict executable uploads
- Use JWT expiration
- Implement refresh tokens
- Store secrets in Azure Key Vault
- Use rate limiting
- Use SQL parameterization
- Enable audit logging
- Use anti-virus scanning

Database Security

- Encrypted connection strings
- Least privilege SQL accounts

- Daily backups
 - Row-level security
 - Soft delete strategy
-

MAUI Security

- Store JWT securely
 - Use SecureStorage
 - Do not store passwords locally
 - Implement auto logout
 - Use certificate pinning
-

19. Deployment Architecture

Production Architecture

```
Internet
|
Azure Front Door / Nginx
|
Load Balancer
|
ASP.NET Core API Servers
|
SQL Server Cluster
|
Azure Blob Storage / NAS
```

20. Deployment Steps

API Deployment

```
dotnet publish -c Release
```

Deploy to:

- IIS
 - Azure App Service
 - Docker Container
 - Kubernetes
-

SQL Server

- Create production database
 - Apply migrations
 - Configure firewall
 - Configure backup jobs
-

MAUI App

Publish targets:

- Windows
 - Android
 - iOS
 - macOS
-

21. Recommended NuGet Packages

API

```
Microsoft.EntityFrameworkCore.SqlServer  
Microsoft.AspNetCore.Authentication.JwtBearer  
Microsoft.AspNetCore.SignalR  
FluentValidation  
Serilog.AspNetCore  
AutoMapper.Extensions.Microsoft.DependencyInjection
```

MAUI

```
MudBlazor  
CommunityToolkit.Maui  
Microsoft.AspNetCore.SignalR.Client
```

22. Enterprise Enhancements

Recommended Future Features

- OCR integration
- Elasticsearch full-text search
- AI document tagging

- Version control
 - File previewing
 - Multi-tenancy
 - SSO integration
 - Azure Blob Storage
 - Background jobs using Hangfire
 - Redis caching
 - CQRS with MediatR
-

23. Recommended Development Phases

Phase 1

- Database
- Authentication
- User management
- Basic upload

Phase 2

- Folder explorer
- SignalR updates
- Logging
- Dashboard analytics

Phase 3

- Advanced permissions
 - Search engine
 - Cloud storage
 - OCR
-

24. Suggested Development Workflow

Backend

1. Create Domain Layer
2. Configure EF Core
3. Create Repositories
4. Build Services
5. Build Controllers
6. Add Authentication
7. Add SignalR
8. Add Logging

Frontend

1. Configure MAUI Blazor
 2. Add Authentication State Provider
 3. Build Layouts
 4. Create Dashboard
 5. Build File Explorer
 6. Add Real-Time Notifications
 7. Add Responsive Design
-

25. Final Enterprise Recommendations

Architecture

- Keep UI independent from API
- Use DTOs only between layers
- Never expose EF entities directly
- Separate commands and queries

Performance

- Use pagination
- Use async everywhere
- Use caching
- Optimize SQL indexes

Maintainability

- Use SOLID principles
- Use dependency injection
- Use centralized exception handling
- Use structured logging

Scalability

- Stateless API
 - Distributed cache
 - Blob storage
 - Containerization
-

26. Recommended Next Step

Implement the solution in this exact order:

1. Solution structure
2. Domain entities

3. Database
4. Authentication
5. User management
6. File upload
7. Folder explorer
8. SignalR
9. MAUI UI
10. Deployment